AI Workfinder

Dokumentacja aplikacji

Autorzy: Roman Nadkernychnyi, Maksim Pyanin, Bohdan Hrytsai, Maksym Stepaniuk

Data opracowania: 30.03.2026

Dokument opisuje problem biznesowy, analizę wymagań, projekt rozwiązania, architekturę, technologie, interfejsy, model danych oraz implementację aplikacji AI Workfinder. Ostatnią część dokumentu stanowi pełna, niezmodyfikowana treść raportu z testów.

Spis treści

- 1. Wprowadzenie
- 2. Opis problemu
- 3. Projekt i analiza
- 4. Implementacja
- 5. Testy – pełna treść raportu z pliku „AI Workfinder raport z testow.pdf”

1. Wprowadzenie

1.1. Cel dokumentu

Celem dokumentu jest przedstawienie kompletnej dokumentacji aplikacji AI Workfinder: zakresu produktu, założeń analitycznych, wymagań funkcjonalnych i pozafunkcjonalnych, modelu danych, interfejsów, architektury oraz implementacji. Dokument ma stanowić spójny opis rozwiązania na potrzeby projektu inżynierskiego/licencjackiego.

1.2. Przyjęte zasady w dokumencie

- Priorytet P0 oznacza wymaganie krytyczne, P1 ważne, P2 uzupełniające.
- Skróty: UI – interfejs użytkownika, API – interfejs programistyczny, JWT – token uwierzytelniający, LLM – model językowy, SSE – Server-Sent Events.
- Opis odnosi się do aktualnej implementacji znajdującej się w repozytorium projektu, a część testowa jest dołączona w postaci pełnego raportu PDF.

1.3. Zakres produktu

AI Workfinder jest webową aplikacją wspierającą użytkownika indywidualnego w wyszukiwaniu pracy. System umożliwia rejestrację i logowanie, uzupełnienie profilu, dodanie CV, skonfigurowanie parametrów wyszukiwania, uruchomienie wieloagentowego pipeline’u AI oraz przegląd ofert i wysłanych aplikacji.

1.4. Literatura

Pozycja	Zakres wykorzystania
Dokumentacja FastAPI	warstwa API, routingi, zależności, modele wejścia/wyjścia
Dokumentacja React i React Router	warstwa SPA, routing, konteksty i ochrona tras
Dokumentacja MongoDB / PyMongo	trwałe przechowywanie danych użytkowników
Dokumentacja LangChain / LangGraph	orchestracja wieloagentowego pipeline’u AI
Dokumentacja OpenAI-compatible API	konfiguracja modelu LLM i połączenia z backendem AI

2. Opis problemu

2.1. Dokładny opis problemu

Poszukiwanie pracy w branży IT wymaga równoległego przeglądania wielu ofert, oceny ich dopasowania do kompetencji użytkownika oraz ręcznego wypełniania formularzy aplikacyjnych. Proces jest czasochłonny, powtarzalny i podatny na pominięcie atrakcyjnych ofert. AI Workfinder ma ograniczyć ten koszt operacyjny poprzez centralizację profilu użytkownika oraz automatyzację wyszukiwania i części procesu aplikowania.

Projekt ma charakter proof of concept / prototypu systemu AI-assisted job search. Głównym celem było sprawdzenie możliwości zbudowania pipeline'u złożonego z wyszukiwania ofert, filtrowania 1-click apply oraz automatycznego wysyłania zgłoszeń na podstawie danych użytkownika i przesłanego CV.

2.2. Porównanie dostępnych rozwiązań

Kategoria	Charakterystyka	Ograniczenia względem AI Workfinder
Ręczne przeglądanie portali pracy	użytkownik sam filtruje i aplikuje	wysoki nakład czasu, brak automatyzacji i centralnego status
Klasyczne agregatory ofert	prezentują oferty i podstawowe filtry	zwykle nie automatyzują wysyłki aplikacji
Rozszerzenia auto-apply / skrypty	wypełniają pojedyncze formularze	zwykle nie mają profilu użytkownika, dashboardu i pełnego p
AI Workfinder	łączy profil, AI search, filtrację i monitoring ofert	ograniczony do jednego źródła ofert i do roli użytko

2.3. Możliwości zastosowania praktycznego

- Wsparcie kandydata IT w szybkim reagowaniu na nowe oferty pracy.
- Automatyzacja rutynowych zadań: wyszukiwania, filtrowania i części procesu aplikowania.
- Budowa podstawy pod przyszły system rekomendacyjny lub platformę job-matching.
- Walidacja użyteczności integracji LLM + crawler + HTTP apply w jednym produkcie.

3. Projekt i analiza

3.1. Perspektywa produktu

Produkt jest aplikacją klient-serwer. Frontend w React udostępnia interfejs webowy, backend w FastAPI udostępnia REST API i SSE, a MongoDB przechowuje profile użytkowników. Wyszukiwanie i automatyczne aplikowanie realizowane są przez pipeline AI korzystający z modelu LLM oraz narzędzi HTTP/scrapingowych.

3.2. Funkcje produktu

- Rejestracja i logowanie użytkownika z użyciem JWT.
- Przechowywanie i edycja profilu użytkownika.
- Przesyłanie pliku CV do lokalnego magazynu plików backendu.
- Konfiguracja procesu wyszukiwania ofert pracy.
- Wielostopniowy pipeline AI: search -> filter -> apply.
- Strumieniowanie statusu oraz logów procesu wyszukiwania.
- Przegląd listy znalezionych ofert i listy wysłanych aplikacji.
- Dwujęzyczny interfejs użytkownika (PL/EN).

3.3. Ograniczenia

- Aktualna implementacja koncentruje się na użytkowniku indywidualnym; role firma/admin nie są zaimplementowane w interfejsie ani API.
- Źródłem ofert w aktualnej wersji jest portal justjoin.it.
- Status wyszukiwania i lista ofert są przechowywane w pamięci procesu, więc restart serwera czyści stan.
- Uruchomienie pipeline’u AI wymaga zewnętrznego endpointu kompatybilnego z OpenAI oraz poprawnej konfiguracji .env.
- Integracja Gmail jest obecna po stronie zależności i części frontendu, ale nie jest obecnie udostępniona jako kompletna ścieżka backendowa.

3.4. Aktorzy i charakterystyka użytkowników

ID	Nazwa	Opis
GUEST	Gość	Osoba niezalogowana. Może założyć konto lub zalogować się do systemu.
USER	Użytkownik	Osoba zalogowana. Zarządza profilem, CV, uruchamia wyszukiwanie i przegląda
AI_PIPELINE	Pipeline AI	Aktor pomocniczy/techniczny odpowiedzialny za wykonanie procesu search-filte
EXTERNAL_SERVICES	Usługi zewnętrzne	JustJoin.it oraz endpoint LLM/OpenAI-compatible, od których zależy pełna realiza

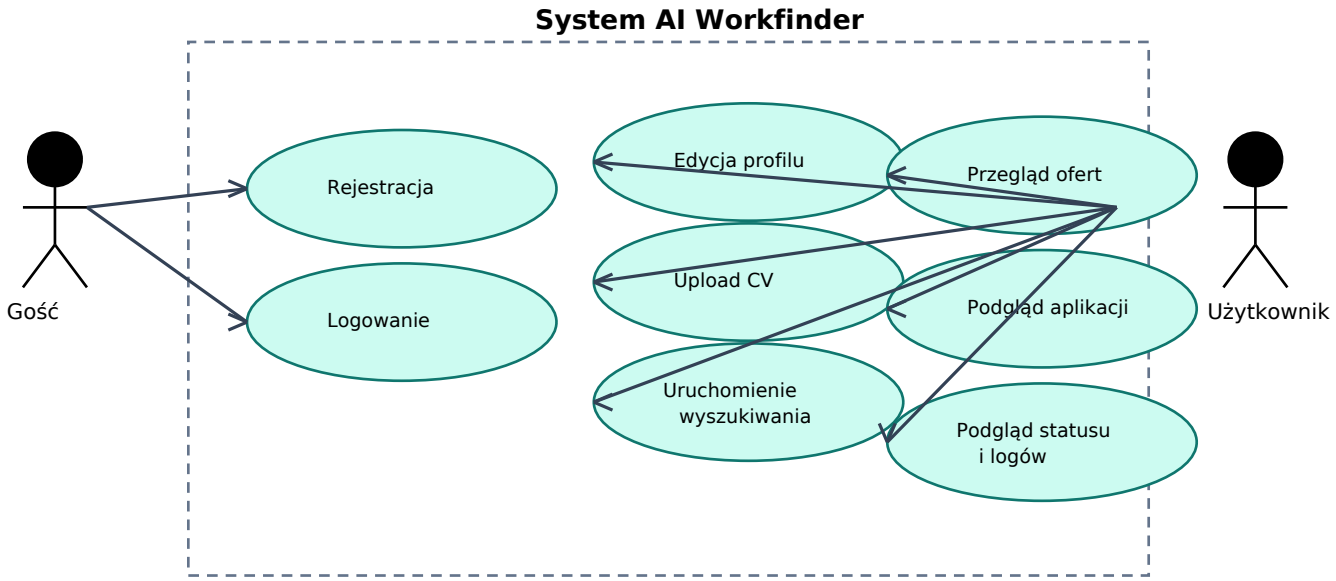
3.5. Obiekty biznesowe

Obiekt	Opis
Profil użytkownika	Dane osobowe i preferencje pracy użytkownika oraz referencja do przesłanego CV.
CV	Plik PDF/DOC/DOCX przechowywany lokalnie po stronie backendu.
Oferta pracy	Opis znalezionego ogłoszenia wraz z URL, źródłem i ewentualnym statusem aplikacji.
Aplikacja	Wynik próby wysłania zgłoszenia do oferty, wraz ze statusem sukces/błąd.

Obiekt	Opis
Stan wyszukiwania	Bieżący status procesu AI, liczniki, zdarzenia i lista wyników.
Konfiguracja uruchomienia	Parametry biegu AI zapisywane do pliku config.json.

3.6. Wymagania funkcjonalne

ID	Wymaganie	Priorytet
FR1	System umożliwia rejestrację użytkownika za pomocą adresu e-mail i hasła.	P0
FR2	System umożliwia logowanie i uwierzytelnianie żądań za pomocą JWT w nagłówku Authorization.	P0
FR3	System udostępnia widok i edycję profilu użytkownika.	P0
FR4	System pozwala przesłać i przechowywać plik CV użytkownika.	P0
FR5	System pozwala uruchomić proces wyszukiwania ofert z parametrami: specjalizacja, poziom, lokalizacja, limity	P0
FR6	System wykonuje pipeline AI obejmujący wyszukiwanie, filtrację ofert 1-click oraz próbę automatycznego apply	P0
FR7	System udostępnia strumień statusu i logów procesu wyszukiwania w czasie zbliżonym do rzeczywistego.	P1
FR8	System prezentuje listę znalezionych ofert pracy oraz listę wysłanych aplikacji.	P0
FR9	Interfejs użytkownika umożliwia zmianę języka między polskim i angielskim.	P1
FR10	System centralizuje komunikację frontend-backend przez wspólny klient HTTP.	P1



Rysunek 1. Diagram przypadków użycia dla AI Workfinder.

3.7. Charakterystyka interfejsów

ID	Wymaganie	Priorytet
UI1	Interfejs musi działać w przeglądarce jako SPA z publicznymi i chronionymi trasami.	P0
UI2	Formularze logowania, rejestracji i profilu muszą walidować obecność wymaganych danych.	P0
UI3	UI musi prezentować stan procesu wyszukiwania, liczby znalezionych ofert i wyniki aplikowania.	P0
UI4	UI powinno aktualizować log procesu bez pełnego odświeżania strony dzięki SSE.	P1
UI5	UI powinno oferować lokalizację PL/EN i spójne nazewnictwo statusów.	P1

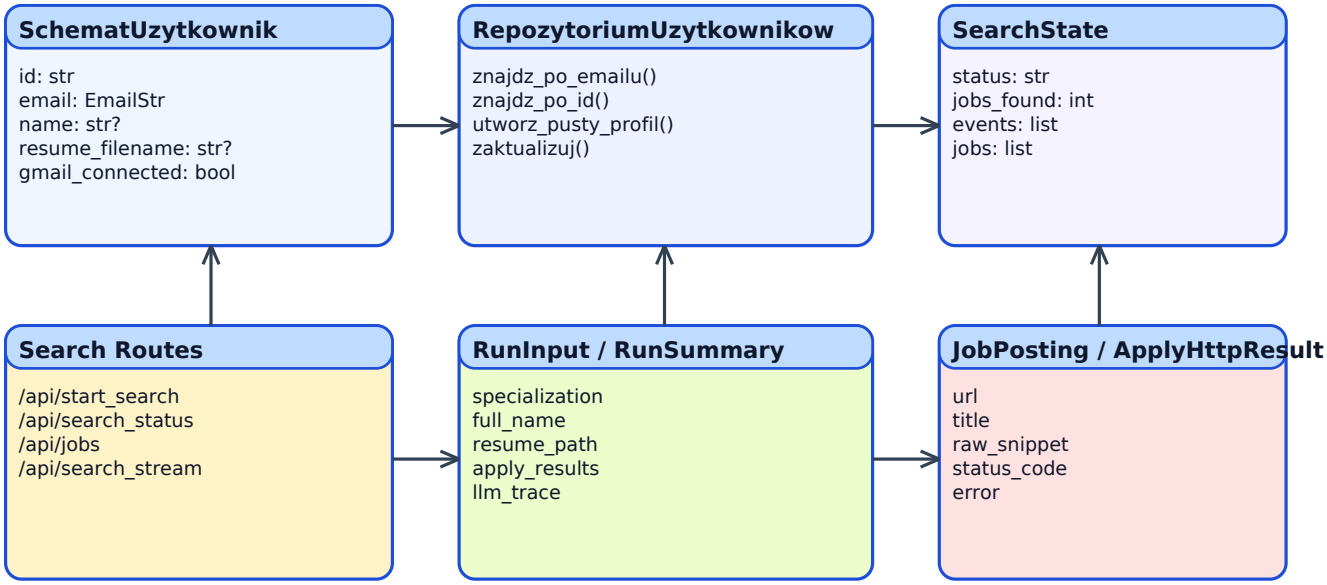
Interfejs	Opis
REST API	Endpointy /api/register, /api/login, /api/profile, /api/upload_resume, /api/start_search, /api/se
SSE	Endpoint /api/search_stream do strumieniowania logów i statusu procesu AI.
MongoDB	Połączenie backendu z bazą dokumentową przechowującą kolekcję users.
OpenAI-compatible API	Backend AI wykorzystuje ChatOpenAI z parametrami z .env i config.json.
JustJoin.it / HTTP apply	Crawler i automatyzacja apply korzystają z zewnętrznych endpointów portalu ofert pracy.

3.8. Wymagania pozafunkcjonalne

ID	Nazwa	Opis	Priorytet
NFR1	Bezpieczeństwo	Hasła nie mogą być przechowywane jako tekst jawny; system stosuje hash PBKDF2 oraz JWT.	P0
NFR2	Integralność danych	Adres e-mail użytkownika musi być unikalny w bazie MongoDB.	P0
NFR3	Użyteczność	Interfejs powinien być prosty, formularzowy i czytelny dla użytkownika nietechnicznego.	P1
NFR4	Obsługa błędów	System powinien zwracać przewidywalne komunikaty walidacyjne i błędy startu wyszukiwania	P0
NFR5	Rozszerzalność	Architektura ma umożliwiać dodanie kolejnych źródeł ofert i kolejnych agentów.	P1
NFR6	Przenośność	Aplikacja powinna działać lokalnie na środowisku developerskim z backendem FastAPI, fronte	P1

3.9. Diagram klas

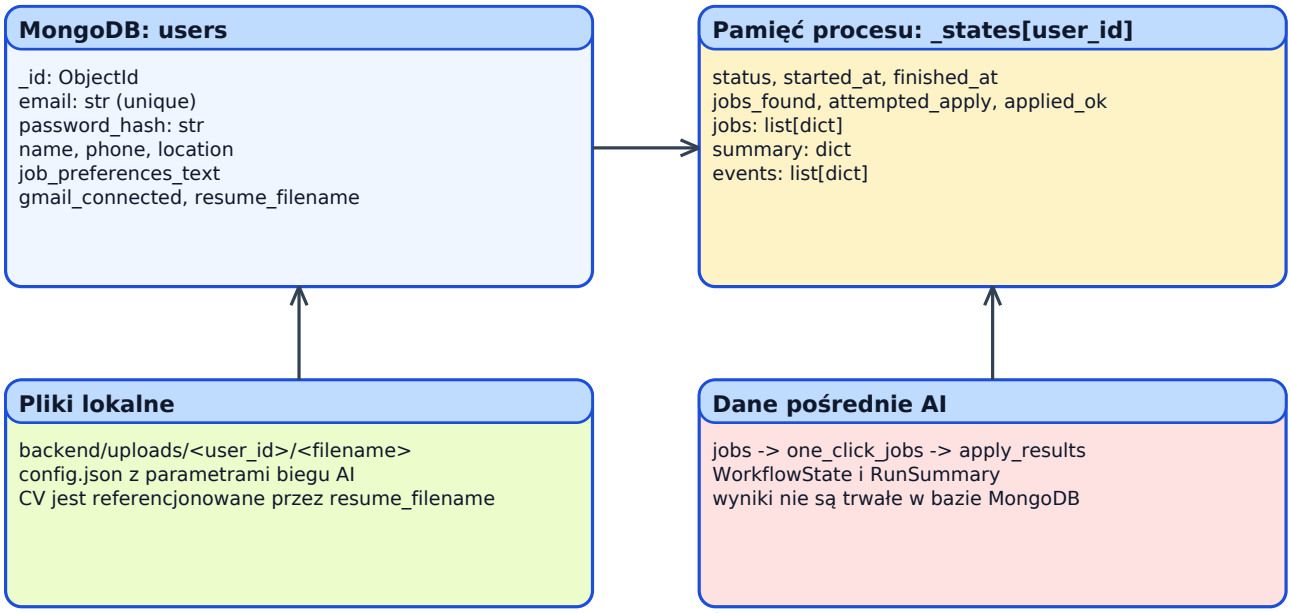
Poniższy diagram pokazuje kluczowe klasy i modele danych biorące udział w działaniu aplikacji. Uwzględniono zarówno warstwę domenową backendu, jak i struktury używane w pipeline'ie AI.



Rysunek 2. Uproszczony diagram klas i modeli systemu.

3.10. Model danych i organizacja przechowywania danych

Projekt nie korzysta z relacyjnej bazy danych. Dane trwałe przechowywane są w dokumentowej bazie MongoDB, natomiast dane operacyjne procesu wyszukiwania są utrzymywane w pamięci aplikacji. CV przechowywane jest w systemie plików backendu, a konfiguracja pojedynczego uruchomienia pipeline'u w pliku config.json.



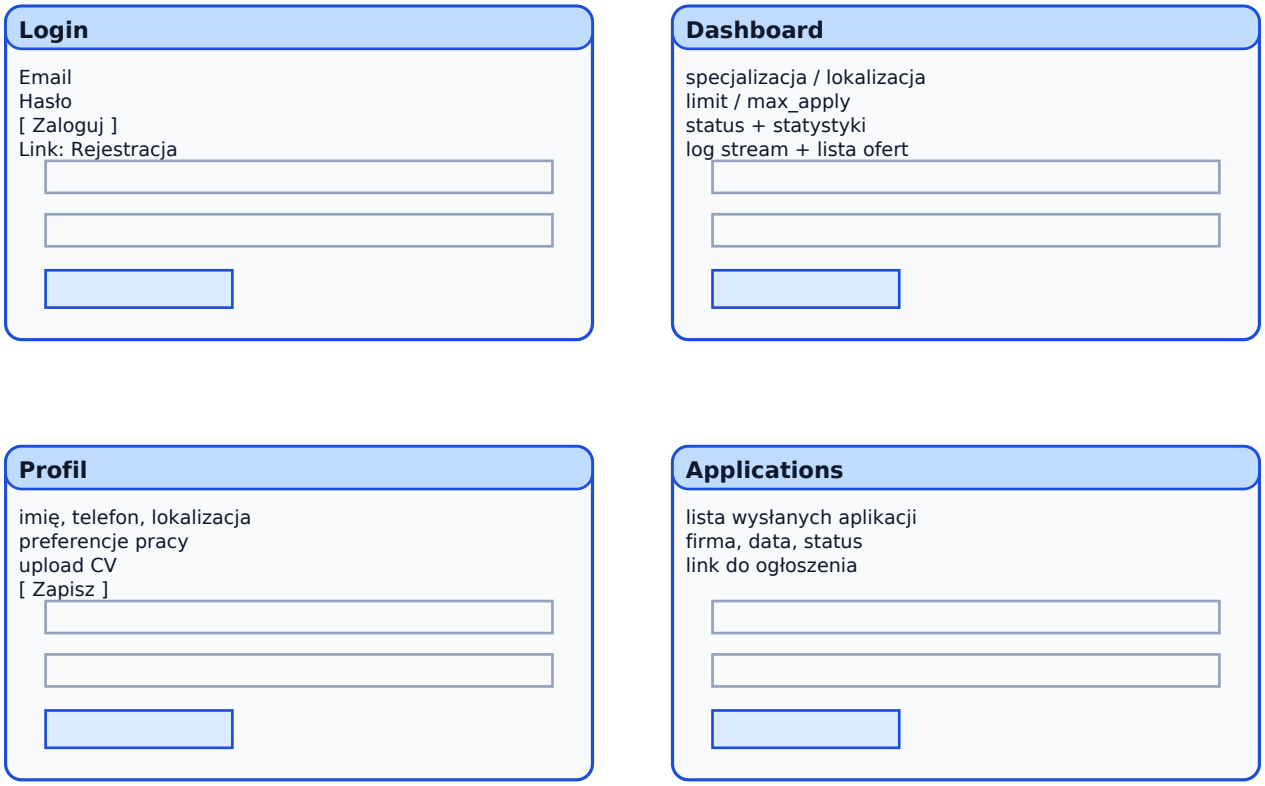
Rysunek 3. Model danych trwałych i danych tymczasowych.

```
Przyk■adowy dokument u■ytkownika (MongoDB / users):
{
  "_id": "ObjectId(...)",
  "email": "user@example.com",
  "password_hash": "<hash>",
  "name": "Jan Kowalski",
  "phone": "+48 123 456 789",
  "location": "Warszawa",
  "job_preferences_text": "Python backend, remote",
  "gmail_connected": false,
  "resume_filename": "resume.pdf"
}

Przyk■adowy stan wyszukiwania (w pami■ci procesu):
{
  "status": "finished",
  "jobs_found": 20,
  "applied_ok": 3,
  "attempted_apply": 3,
  "events": [{"ts": "...", "level": "INFO", "message": "AI search started"}],
  "jobs": [{ "title": "Python Developer", "apply_url": "...", "application_status": "Applied" }]
}
```

3.11. Projekt interfejsu użytkownika

Interfejs został zorganizowany wokół czterech głównych widoków: logowania, panelu wyszukiwania, profilu oraz listy aplikacji. Część zalogowana pracuje w układzie z lewym menu nawigacyjnym.

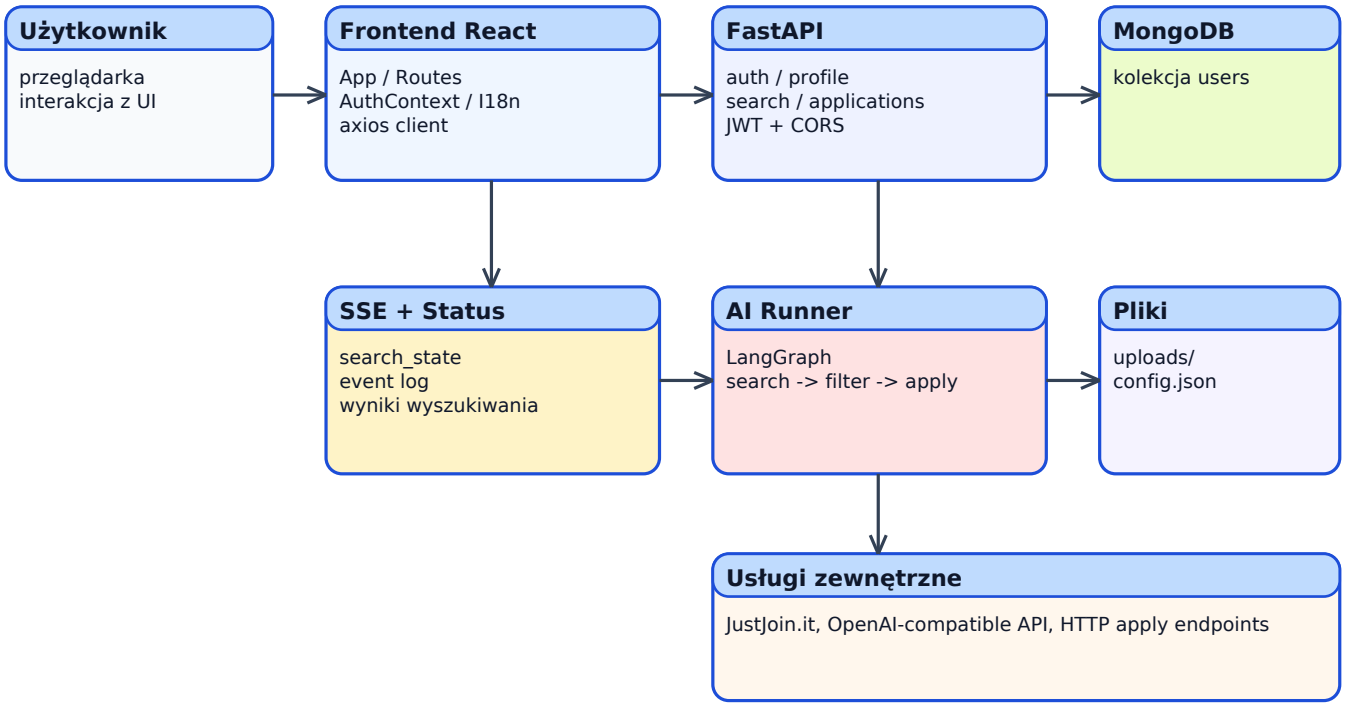


Rysunek 4. Uproszczone makiety kluczowych ekranów aplikacji.

4. Implementacja

4.1. Architektura rozwiązania

Architektura systemu ma postać klient-serwer. Frontend React odpowiada za routing, formularze, lokalny stan sesji i prezentację danych. Backend FastAPI wystawia API HTTP/SSE, realizuje autoryzację i obsługę plików oraz steruje pipeline'em AI. MongoDB przechowuje wyłącznie dane użytkowników. Wyniki wyszukiwania i logi przebiegu są trzymane w pamięci procesu.



Rysunek 5. Diagram architektury rozwiązania.

4.2. Użyte technologie

Warstwa	Technologie
Frontend	React 19, React Router, Vite, axios, CSS, konteksty Auth/I18n
Backend API	Python, FastAPI, Uvicorn, Pydantic
Baza danych	MongoDB, PyMongo
Bezpieczeństwo	JWT (python-jose), Passlib PBKDF2
AI / orkiestracja	langchain-openai, langgraph, ChatOpenAI
Scraping / apply	requests, BeautifulSoup, lxml, własne wrappery HTTP
Pliki i konfiguracja	python-dotenv, lokalny storage backend/uploads, config.json
Testy i raportowanie	pytest, pytest-cov, Vitest, Testing Library, coverage reports

4.3. Moduły backendowe

Moduł	Rola
app/api/routes/auth.py	rejestracja i logowanie użytkownika
app/api/routes/profile.py	pobieranie i aktualizacja profilu, upload CV

Moduł	Rola
app/api/routes/search.py	start procesu AI, status, lista ofert, stream SSE
app/api/routes/applications.py	lista wysłanych aplikacji użytkownika
app/repositories/uzytkownik_repo.py	operacje CRUD na kolekcji users
app/services/search_state.py	stan procesu wyszukiwania w pamięci
app/services/ai_runner.py	walidacja configu, uruchomienie grafu agentów
app/services/ai/*	definicje agentów, grafu i stanów workflow
app/tools/*	crawler, filtr 1-click, wrapper apply HTTP

4.4. Moduły frontendowe

Moduł	Rola
src/App.jsx	definicja tras publicznych i chronionych
src/context/AuthContext.jsx	stan sesji, login, register, logout, pobranie profilu
src/context/I18nContext.jsx	słowniki tłumaczeń PL/EN i zmiana języka
src/pages/LoginPage.jsx / RegisterPage.jsx	formularze wejścia do systemu
src/pages/ProfilePage.jsx	edycja profilu i upload CV
src/pages/DashboardPage.jsx	konfiguracja wyszukiwania, SSE, lista ofert
src/pages/ApplicationsPage.jsx	lista wysłanych aplikacji
src/components/MainLayout.jsx	szkielet części zalogowanej
src/api/*.js	warstwa komunikacji z backendem

4.5. Integracje zewnętrzne i ograniczenia implementacyjne

- Endpoint LLM jest dostarczany zewnętrznie i wskazywany przez OPENAI_BASE_URL oraz OPENAI_API_KEY.
- Część apply HTTP zależy od kompatybilności z aktualnym formularzem i endpointami justjoin.it.
- Magazyn aplikacji nie jest trwały; po restarcie backendu stan wyszukiwania i lista ofert są tracone.
- W kodzie przygotowano zależności pod Celery/Redis i Gmail API, lecz w aktualnej implementacji główna ścieżka działa na FastAPI BackgroundTasks i lokalnym pipeline’ie.

5. Testy

Poniżej dołączono pełną, niezmodyfikowaną treść pliku „AI Workfinder raport z testow.pdf”. Raport stanowi ostatnią część dokumentacji zgodnie z wymaganiem projektowym.

AI Workfinder

Roman Nadkernychnyi
Maksim Pyanin
Bohdan Hrytsai
Maksym Stepaniuk

1 Testy

Aplikacja została poddana testom automatycznym obejmującym zarówno warstwę backendową, jak i frontendową. Celem przeprowadzonych testów była weryfikacja poprawności działania kluczowych modułów systemu, sprawdzenie logiki biznesowej, obsługi błędów, poprawności przetwarzania danych wejściowych oraz stabilności interfejsów aplikacyjnych. Testy miały również na celu potwierdzenie bezpieczeństwa wybranych mechanizmów systemu oraz ograniczenie ryzyka występowania błędów regresyjnych w kolejnych etapach rozwoju projektu.

1.1 Zakres testów

Zakres testów objął komponenty backendowe i frontendowe projektu. Struktura testów została rozdzielona zgodnie z architekturą aplikacji i rolą poszczególnych modułów.

1.1.1 Struktura testów

Testy backendowe znajdują się w katalogach:

- `backend/tests/unit`,
- `backend/tests/integration`.

W ramach testów backendowych wyróżniono następujące sekcje:

- `unit/core`,
- `unit/repositories`,
- `unit/services`,
- `unit/services/ai`,
- `unit/tools`,
- `integration/api/routes`.

Testy frontendowe znajdują się w katalogu:

- `job-agent-frontend/src/test`.

W warstwie frontendowej wyróżniono następujące sekcje:

- `node/api`,
- `node/smoke`,

- `dom/app`,
- `dom/components`,
- `dom/contexts`,
- `dom/pages`,
- `dom/router`.

1.1.2 Liczba testów

Łącznie w repozytorium przygotowano 130 testów automatycznych, z czego:

- 89 testów dotyczy backendu,
- 41 testów dotyczy frontendu.

W obrębie backendu wydzielono:

- `unit/core` - 4 testy,
- `unit/repositories` - 9 testów,
- `unit/services` - 12 testów,
- `unit/services/ai` - 13 testów,
- `unit/tools` - 24 testy,
- `integration/api/routes` - 27 testów.

W obrębie frontendu wyróżniono:

- `node/api` - 10 testów,
- `node/smoke` - 4 testy,
- `dom/app` - 2 testy,
- `dom/components` - 4 testy,
- `dom/contexts` - 5 testów,
- `dom/pages` - 13 testów,
- `dom/router` - 3 testy.

1.1.3 Wykorzystane biblioteki i narzędzia

Do testowania warstwy backendowej wykorzystano następujące narzędzia:

- `pytest`,
- `pytest-cov`,
- `httpx`,
- `fastapi.testclient`.

Do testowania warstwy frontendowej wykorzystano:

- `vitest`,
- `@testing-library/react`,
- `@testing-library/jest-dom`,
- `@testing-library/user-event`,
- `happy-dom`,
- `@vitest/coverage-v8`.

1.1.4 Zakres przetestowanych przypadków

W warstwie backendowej przetestowano przede wszystkim:

- bezpieczeństwo: haszowanie hasła, weryfikację hasła, obsługę tokenów JWT, przypadki tokenu błędnego i wygasłego,
- repozytorium użytkownika: operacje CRUD, pobieranie po adresie e-mail i identyfikatorze, pusty update, lazy collection oraz indeksowanie,
- profil użytkownika: odczyt danych, aktualizację profilu, upload CV, pusty plik, błędny format, brak nazwy pliku oraz plik większy niż 5 MB,
- mechanizmy autoryzacji: rejestrację, duplikat adresu e-mail, poprawny i błędny login,
- moduł wyszukiwania: walidację danych wejściowych, brak CV, brak pełnej nazwy użytkownika, brak pliku CV, pustą specjalizację, blokadę równoległego wyszukiwania, status, stream oraz ścieżki błędów,
- moduł aplikacji: normalizację statusów oraz filtrowanie tylko wysłanych aplikacji,
- moduły AI i narzędzia pomocnicze: parser, crawler, mechanizm one-click apply, apply HTTP, graph flow, runner oraz CLI.

W warstwie frontendowej przetestowano:

- klient HTTP: auth header, custom base URL, operacje GET/POST/PUT, `FormData`, odpowiedzi plain-text i JSON, obsługę błędów oraz interceptory,
- warstwę API wrapperów: `auth`, `jobs`, `applications`, `gmail`,
- warstwę UI i DOM: logowanie, rejestrację, profil, dashboard, widok aplikacji, listę ofert pracy, protected route, auth context oraz i18n.

1.1.5 Wyniki wykonania testów backendu

Potwierdzony wynik wykonania backendowych testów automatycznych był następujący:

- środowisko: Python 3.13.2, pytest 9.0.2,
- wynik: 89 passed, 0 failed, 0 skipped,
- czas wykonania: 9.07 s,
- liczba ostrzeżeń: 532,
- pokrycie kodu: 95%,
- podsumowanie coverage: 1126 statements, 52 missed.

1.1.6 Wyniki wykonania testów frontend HTTP/API

Potwierdzony wynik wykonania testów frontendowych dla warstwy HTTP/API przedstawiał się następująco:

- wynik: 14 passed,
- liczba plików testowych zakończonych sukcesem: 6,
- pokrycie dla `src/api/**/*.js`:
  - Lines: 100%,
  - Statements: 100%,
  - Functions: 100%,
  - Branches: 91.3%.

1.1.7 Uwagi metodologiczne

Należy zaznaczyć, że backendowe testy integracyjne są głównie testami na poziomie handlerów i tras, a nie pełnymi testami HTTP end-to-end. Warstwa danych nie korzysta podczas testów z rzeczywistej bazy danych, lecz ze stubowanej `FakeCollection` działającej w pamięci. Dodatkowo w projekcie istnieje fixture oparta o `TestClient`, lecz obecnie nie jest ona używana bezpośrednio w testach.

1.1.8 Raporty testowe

W projekcie przygotowano następujące raporty:

- backendowy raport HTML z testów: `backend/reports/tests/backend-tests.html`,
- backendowy raport HTML pokrycia kodu: `backend/reports/coverage/index.html`,
- frontendowy raport HTML pokrycia kodu: `job-agent-frontend/coverage/index.html`,
- frontendowe podsumowanie coverage w formacie JSON: `job-agent-frontend/coverage/coverage-`

1.2 Kluczowe przypadki testowe

W procesie przygotowywania testów wyodrębniono najważniejsze przypadki testowe obejmujące zarówno logikę backendową, jak i wybrane elementy frontendu. Przypadki te pozwalają zweryfikować poprawność działania aplikacji w scenariuszach standardowych, błędnych oraz brzegowych.

ID	Opis	Wynik
UT1	Haszowanie hasła i poprawna weryfikacja wygenerowanego skrótu.	Sukces
UT2	Odrzucenie błędnego lub wygasłego tokenu JWT.	Sukces
UT3	Utworzenie użytkownika w repozytorium i pobranie po adresie e-mail.	Sukces
UT4	Obsługa pustego update użytkownika bez uszkodzenia danych.	Sukces
UT5	Poprawna obsługa lazy collection i indeksu dla repozytorium użytkownika.	Sukces
UT6	Walidacja przejść stanu procesu wyszukiwania oraz zapisu event log.	Sukces
UT7	Poprawne uruchomienie modułu AI runner i graph flow.	Sukces
UT8	Normalizacja danych wejściowych przez parser.	Sukces
UT9	Obsługa wyjątku w narzędziu apply HTTP.	Sukces
UT10	Test działania crawlera oraz one-click apply.	Sukces
IT1	Rejestracja użytkownika z poprawnymi danymi.	Sukces
IT2	Próba rejestracji z duplikatem adresu e-mail.	Błąd obsłużony poprawnie
IT3	Logowanie z poprawnymi danymi uwierzytelniającymi.	Sukces
IT4	Logowanie z błędnym hasłem.	Błąd autoryzacji obsłużony poprawnie
IT5	Pobranie i aktualizacja profilu użytkownika.	Sukces
IT6	Przesłanie pustego pliku CV.	Błąd walidacyjny obsłużony poprawnie
IT7	Przesłanie pliku CV w nieobsługiwanym formacie.	Błąd walidacyjny obsłużony poprawnie
IT8	Przesłanie pliku większego niż 5 MB.	Błąd walidacyjny obsłużony poprawnie
IT9	Uruchomienie wyszukiwania bez wymaganego CV.	Błąd walidacyjny obsłużony poprawnie
IT10	Zablokowanie równoległego uruchomienia procesu wyszukiwania.	Sukces
IT11	Pobranie statusu i stream dla procesu wyszukiwania.	Sukces
FT1	Dodanie auth header w kliencie HTTP frontendu.	Sukces
FT2	Obsługa odpowiedzi JSON i plain-text w warstwie API.	Sukces
FT3	Test wrapperów API dla modułów auth, jobs, applications i gmail.	Sukces

FT4	Renderowanie widoków login, register i profile.	Sukces
FT5	Działanie protected route oraz auth context.	Sukces

Tabela 1: Przykładowe przypadki testowe

1.3 Podsumowanie wyników testów automatycznych

1.3.1 Statystyki testów backendowych

Tabela 2: Statystyki testów backendowych

Sekcja	**Liczba testów**	**Stan**
`unit/core`	4	100% sukces
`unit/repositories`	9	100% sukces
`unit/services`	12	100% sukces
`unit/services/ai`	13	100% sukces
`unit/tools`	24	100% sukces
`integration/api/routes`	27	100% sukces
Razem	**89**	**100% sukces**

1.3.2 Statystyki testów frontendowych

Tabela 3: Statystyki testów frontendowych

Sekcja	**Liczba testów**	**Stan**
`node/api`	10	przygotowane
`node/smoke`	4	przygotowane
`dom/app`	2	przygotowane
`dom/components`	4	przygotowane
`dom/contexts`	5	przygotowane
`dom/pages`	13	przygotowane
`dom/router`	3	przygotowane
Razem	**41**	**częściowo potwierdzone**

1.3.3 Statystyki ogólne projektu

Tabela 4: Zbiorcze statystyki testów w projekcie

Kategoria	**Liczba testów**	**Stan**
Backend	89	potwierdzone
Frontend	41	częściowo potwierdzone
Łącznie	**130**	**projekt objęty szerokim zakresem testów**

1.3.4 Pokrycie kodu

Backend osiągnął 95% pokrycia kodu przy 1126 analizowanych instrukcjach i 52 niewykonanych. Wynik ten należy uznać za bardzo wysoki i świadczący o szerokim zakresie automatycznej weryfikacji logiki systemu. W przypadku testów frontendowych potwierdzono pełne pokrycie linii, instrukcji i funkcji dla warstwy `src/api/**/*.js`, przy jednoczesnym pokryciu gałęzi na poziomie 91.3%.

1.3.5 Frontend

All files

100% Statements 45/45 **73.91%** Branches 17/23 **100%** Functions 15/15 **100%** Lines 45/45

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements		Branches		Functions		Lines	
applications.js		100%	1/1	100%	0/0	100%	1/1	100%	1/1
auth.js		100%	4/4	100%	0/0	100%	4/4	100%	4/4
gmail.js		100%	1/1	100%	0/0	100%	1/1	100%	1/1
http.js		100%	36/36	73.91%	17/23	100%	6/6	100%	36/36
jobs.js		100%	3/3	100%	0/0	100%	3/3	100%	3/3

Rysunek 1: Pokrycie testów frontendowych

1.3.6 Backend

Coverage report: 95%

Files | Functions | Classes

filter...

hide covered

File ▲	statements	missing	excluded	coverage
backend / app / api / deps.py	19	0	0	100%
backend / app / api / routes / applications.py	34	0	0	100%
backend / app / api / routes / auth.py	29	0	0	100%
backend / app / api / routes / profile.py	43	2	0	95%
backend / app / api / routes / search.py	156	10	0	94%
backend / app / core / config.py	11	0	0	100%
backend / app / core / security.py	23	0	0	100%
backend / app / db / mongo.py	12	0	0	100%
backend / app / main.py	13	1	0	92%
backend / app / repositories / uzytkownik_repo.py	25	0	0	100%
backend / app / schemas / uzytkownik.py	21	0	0	100%
backend / app / services / ai / agents.py	123	13	0	89%
backend / app / services / ai / graph_builder.py	16	0	0	100%
backend / app / services / ai / main.py	46	2	0	96%
backend / app / services / ai / run_from_config.py	36	4	0	89%
backend / app / services / ai / state.py	61	0	0	100%
backend / app / services / ai_runner.py	29	0	0	100%
backend / app / services / paths.py	11	1	0	91%
backend / app / services / search_state.py	82	0	0	100%
backend / app / tools / apply_http_tool.py	111	8	0	93%
backend / app / tools / apply_http_wrapper.py	47	1	0	98%
backend / app / tools / one_click_apply_tool.py	29	5	0	83%
backend / app / tools / one_click_apply_wrapper.py	20	0	0	100%
backend / app / tools / parser_crawler_wrapper.py	22	0	0	100%
backend / app / tools / parser_crawler.py	107	5	0	95%
Total	**1126**	**52**	**0**	**95%**

Rysunek 2: Pokrycie testów backendowych

1.4 Wnioski

Przeprowadzone testy potwierdziły poprawność działania kluczowych elementów backendu i istotnej części frontendu. Szczególnie ważne było zweryfikowanie mechanizmów bezpieczeństwa,

obsługi błędnych danych wejściowych, walidacji plików CV, logiki wyszukiwania, przetwarzania danych aplikacyjnych oraz działania modułów AI i narzędzi pomocniczych.

Wszystkie uruchomione testy backendowe zakończyły się powodzeniem, co wskazuje na wysoką stabilność tej części systemu. Osiągnięte 95% pokrycia kodu backendu potwierdza, że automatyczna weryfikacja objęła zdecydowaną większość najważniejszych fragmentów aplikacji. Dodatkowo potwierdzone testy frontendowe dla warstwy HTTP/API wykazały bardzo wysoki poziom pokrycia kodu w module komunikacji z API.